



# AI IN ACTION

Đào tạo Nhân tài AI thực chiến

**MCP & A2A Infrastructure**



Knowledge — Skills — Lộ trình 5 phần

## Kiến thức (Knowledge)

- Phân biệt monolithic LLM vs multi-agent system
- Khái niệm A2A: AgentCard, Task, Message, Part, Artifact
- Vai trò của MCP — bổ trợ A2A như thế nào
- Khi nào dùng A2A vs khi nào dùng MCP

## Kỹ năng (Skills)

- Đọc và triển khai AgentCard JSON đúng chuẩn
- Dùng LangGraph StateGraph + Send API để fan-out song song
- Cấu hình service discovery động qua Registry
- Trace một yêu cầu xuyên qua nhiều agent với trace\_id

## Lộ trình 5 phần

**Phần 1.** Bối cảnh — vì sao multi-agent? Lộ trình tiến hoá LLM

**Phần 2.** MCP — chuẩn cho LLM kết nối công cụ và dữ liệu

**Phần 3.** A2A — chuẩn cho các agent giao tiếp với nhau

**Phần 4.** So sánh A2A vs MCP — khi nào dùng cái nào?

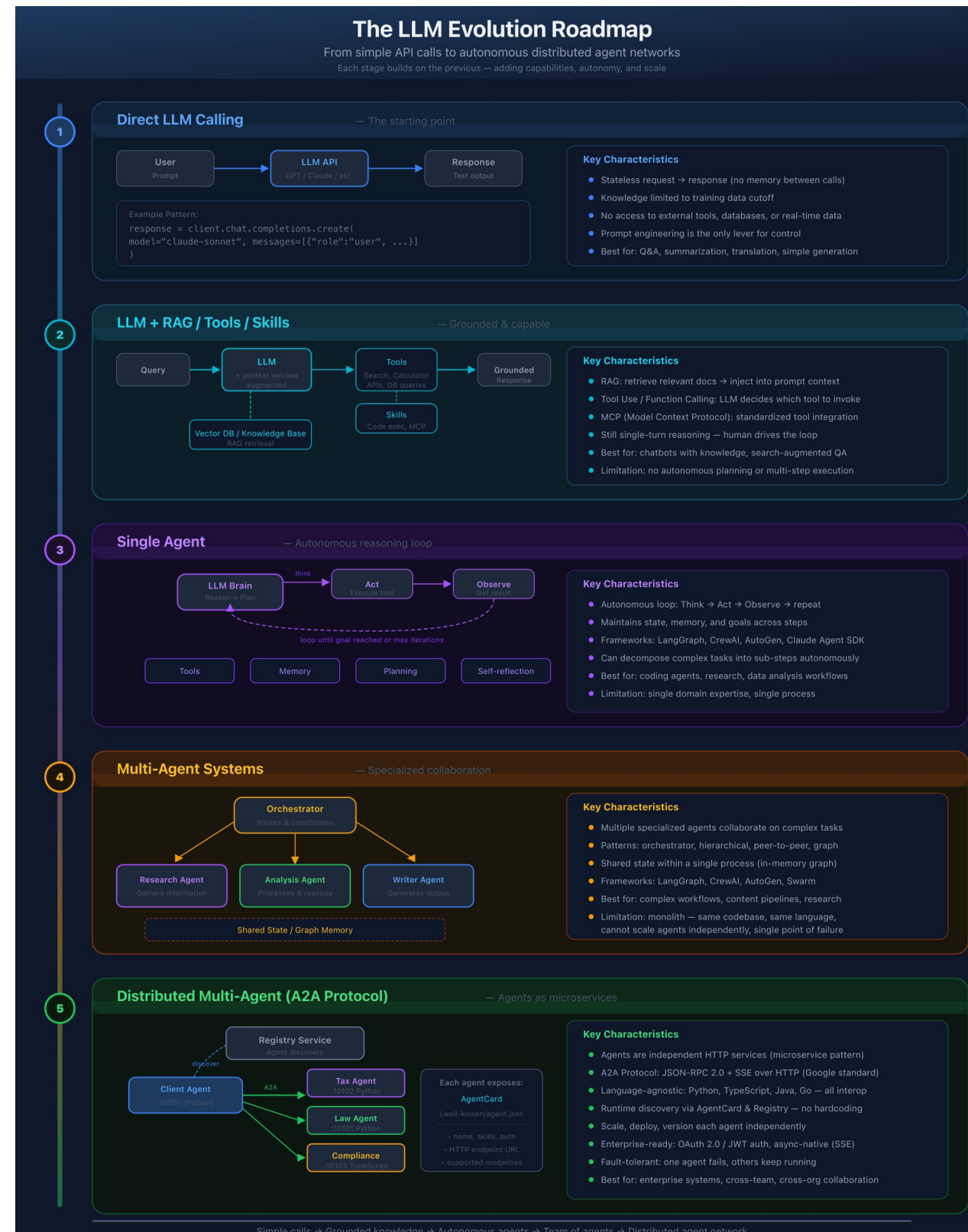
**Phần 5.** Case study: hệ thống tư vấn pháp lý phân tán

# Lộ trình tiến hoá LLM (Stage 1 → 5)

## AI IN ACTION

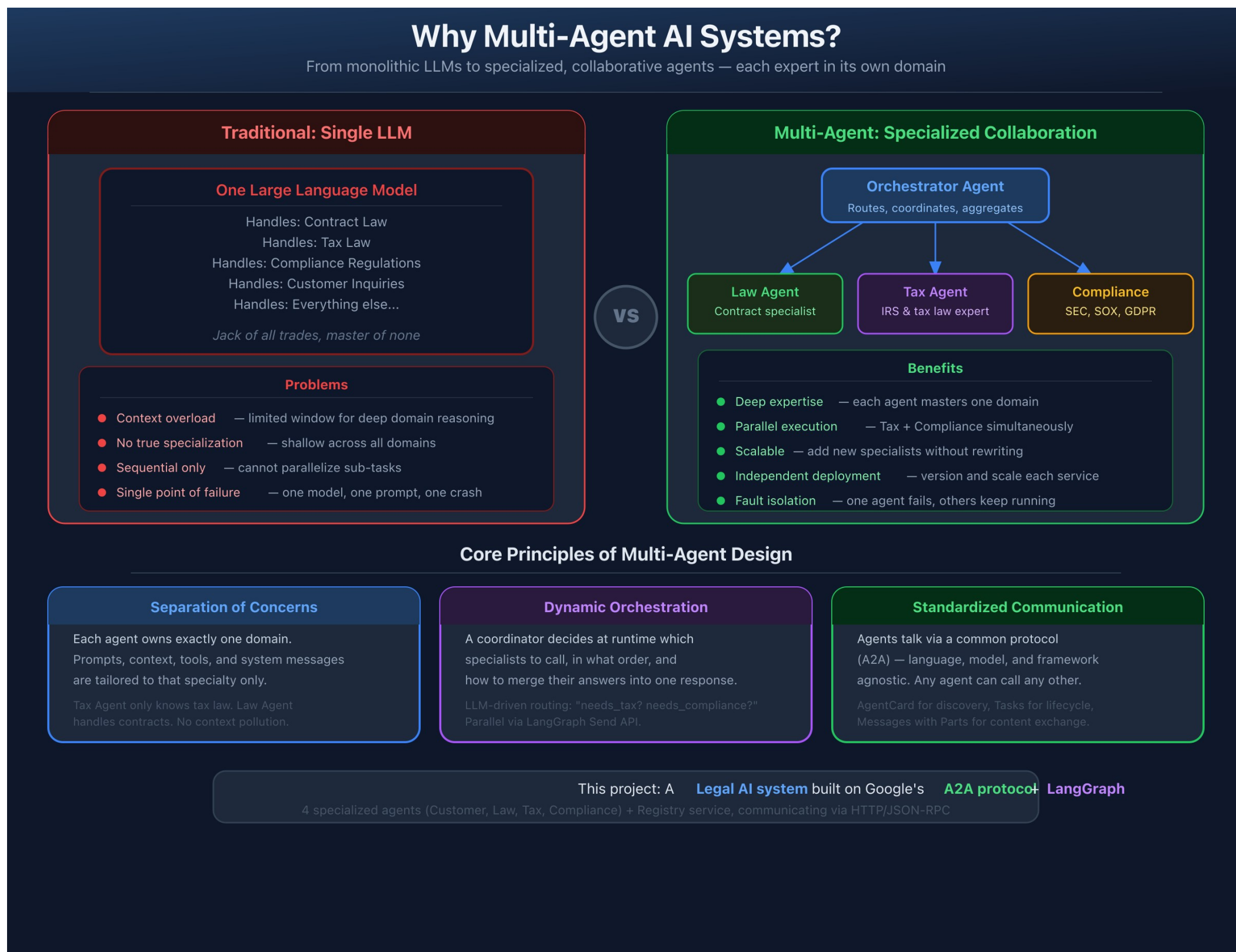
Đào tạo Nhân tài AI thực chiến

Từ gọi API đơn giản đến mạng lưới agent phân tán



# Vì sao cần Multi-Agent?

Hạn chế của LLM đơn lẻ và lợi thế của hệ chuyên gia



# Hai chuẩn mới của thế hệ AI Agent

AI IN ACTION  
Đào tạo Nhân tài AI thực chiến

A2A và MCP — bổ sung lẫn nhau, không thay thế

## MCP — Model Context Protocol

Do **Anthropic** giới thiệu (2024)

“USB-C cho AI” — chuẩn để LLM kết nối với nguồn dữ liệu và công cụ.

- Client-Server architecture
- Server cung cấp: resources, tools, prompts
- LLM (client) gọi tool, đọc file, query DB qua chuẩn chung
- Quan hệ: LLM ↔ Tool / Data

## A2A — Agent-to-Agent Protocol

Do **Google** giới thiệu (2025)

Chuẩn để các agent độc lập giao tiếp như những đối tác bình đẳng.

- Peer-to-peer architecture
- Server công bố: AgentCard, skills, tasks
- Agent có thể uỷ thác (delegate) công việc cho agent khác
- Quan hệ: Agent ↔ Agent

### Phép loại suy

MCP ≈ **USB-C** → 1 thiết bị (LLM) cắm với nhiều phụ kiện (DB, API, file).

A2A ≈ **TCP/IP** → nhiều máy độc lập trên mạng nói chuyện với nhau.

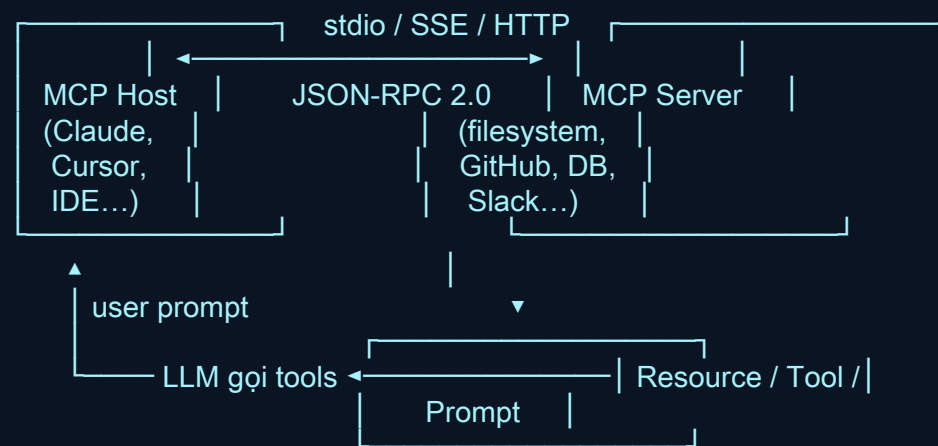
 **Thực tế dùng cả hai:** một agent A2A bên trong vẫn có thể dùng nhiều MCP server để truy cập dữ liệu.

Chuẩn hoá kết nối giữa LLM với thế giới bên ngoài

## Vấn đề MCP giải quyết

**Trước MCP:** mỗi ứng dụng AI tự viết tích hợp riêng cho từng công cụ. Bài toán  $N \times M$ : **N ứng dụng**  $\times$  **M công cụ** =  $N \times M$  tích hợp.

**Với MCP:** mỗi công cụ chỉ cần một **MCP Server**, mỗi ứng dụng AI chỉ cần một **MCP Client**. Tổng tích hợp giảm xuống  $N + M$ .



### 3 thành phần cốt lõi của MCP Server

- **Resources** — dữ liệu chỉ đọc (file, log, snapshot DB)
- **Tools** — hàm có side-effect (gửi email, chạy SQL, tạo PR)
- **Prompts** — mẫu prompt tái sử dụng (templates)

### Khi nào dùng MCP

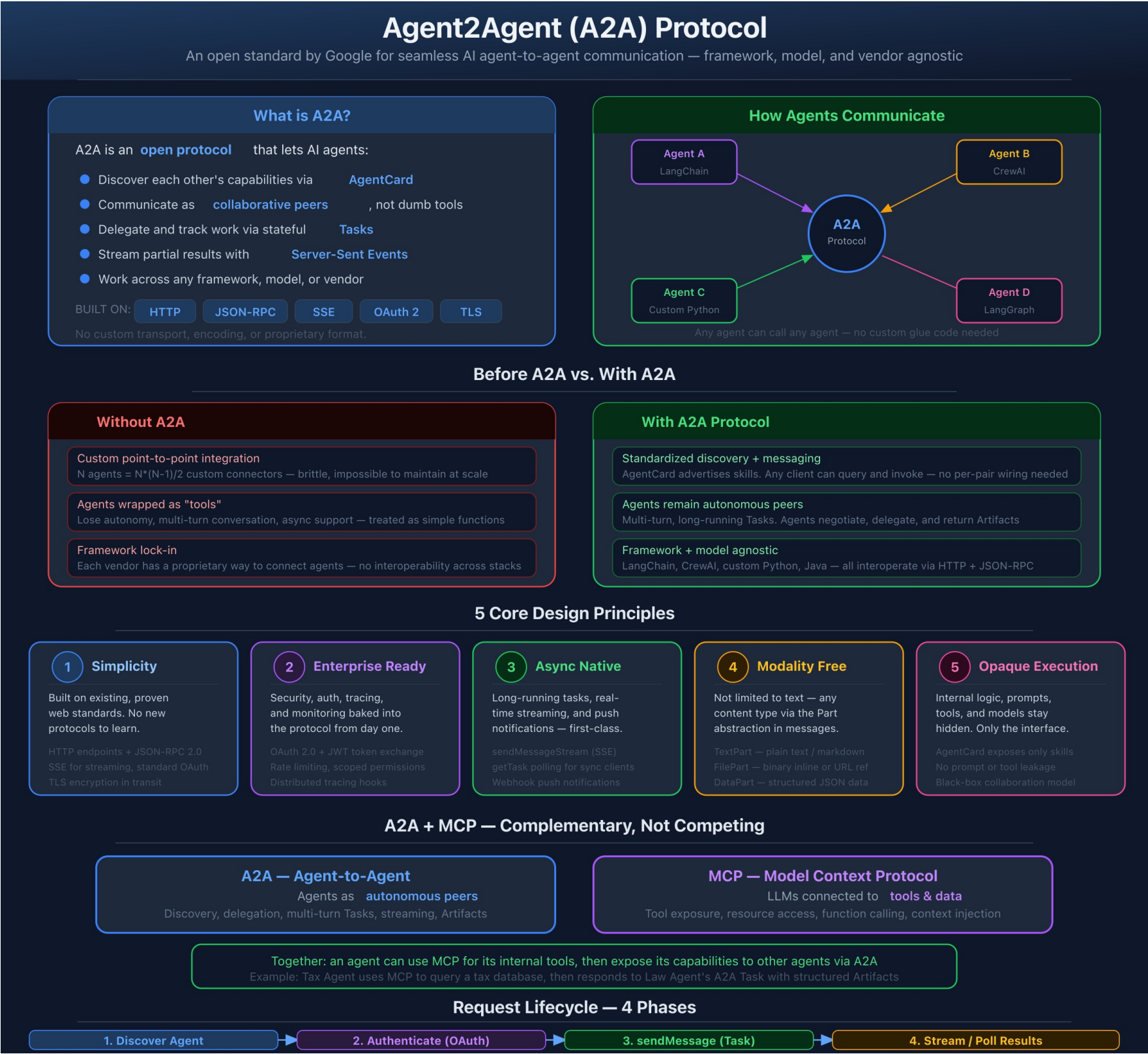
- Cần cho LLM quyền truy cập dữ liệu / công cụ nội bộ
- Muốn tách phần kết nối khỏi logic agent (loose coupling)
- Reuse cùng một MCP server cho nhiều IDE / app khác nhau
- Cần kiểm soát quyền: phê duyệt từng lần gọi tool

# Agent2Agent (A2A) Protocol

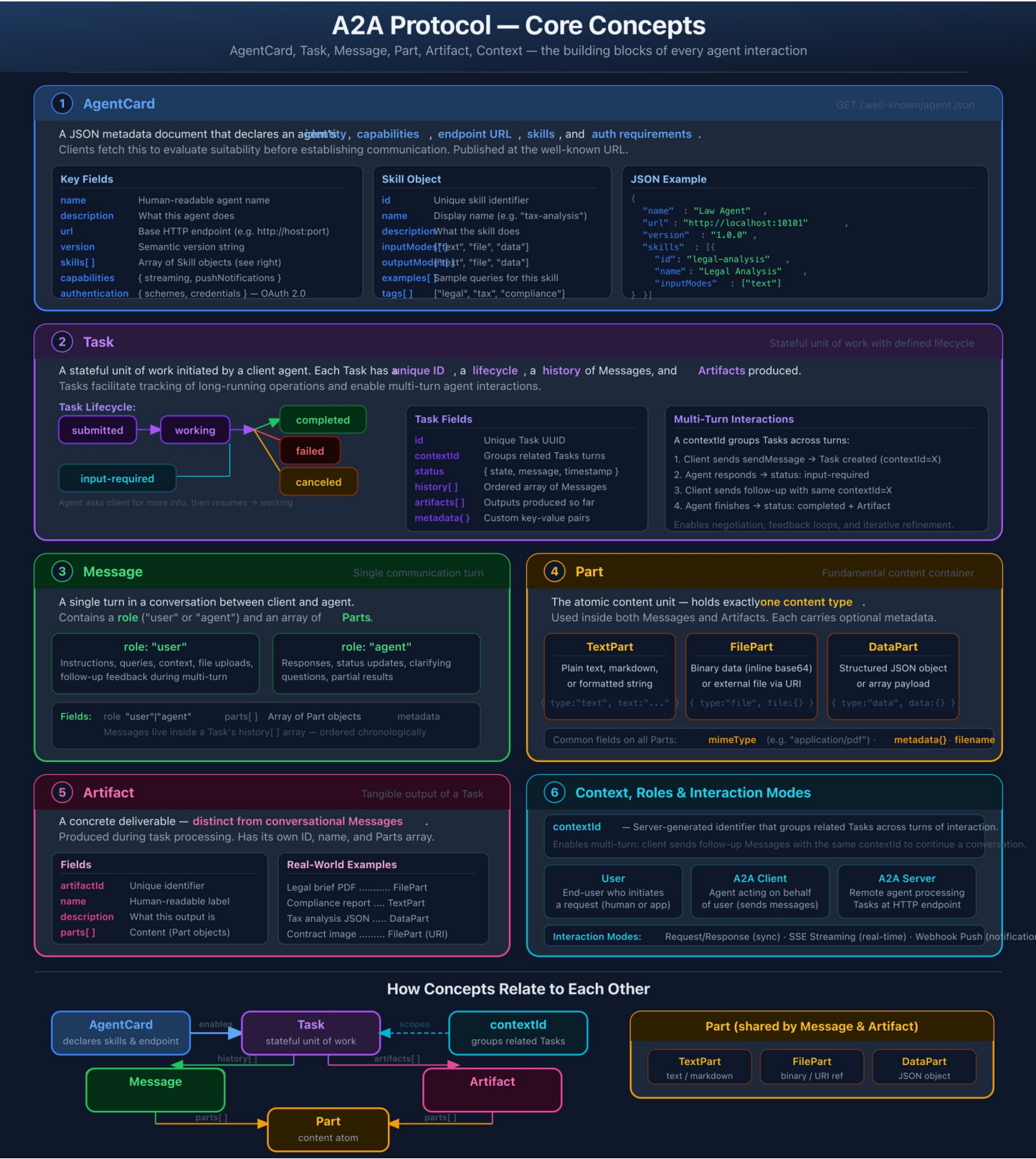
# AI IN ACTION

Đào tạo Nhân tài AI thực chiến

Chuẩn mở của Google cho giao tiếp giữa các AI agent

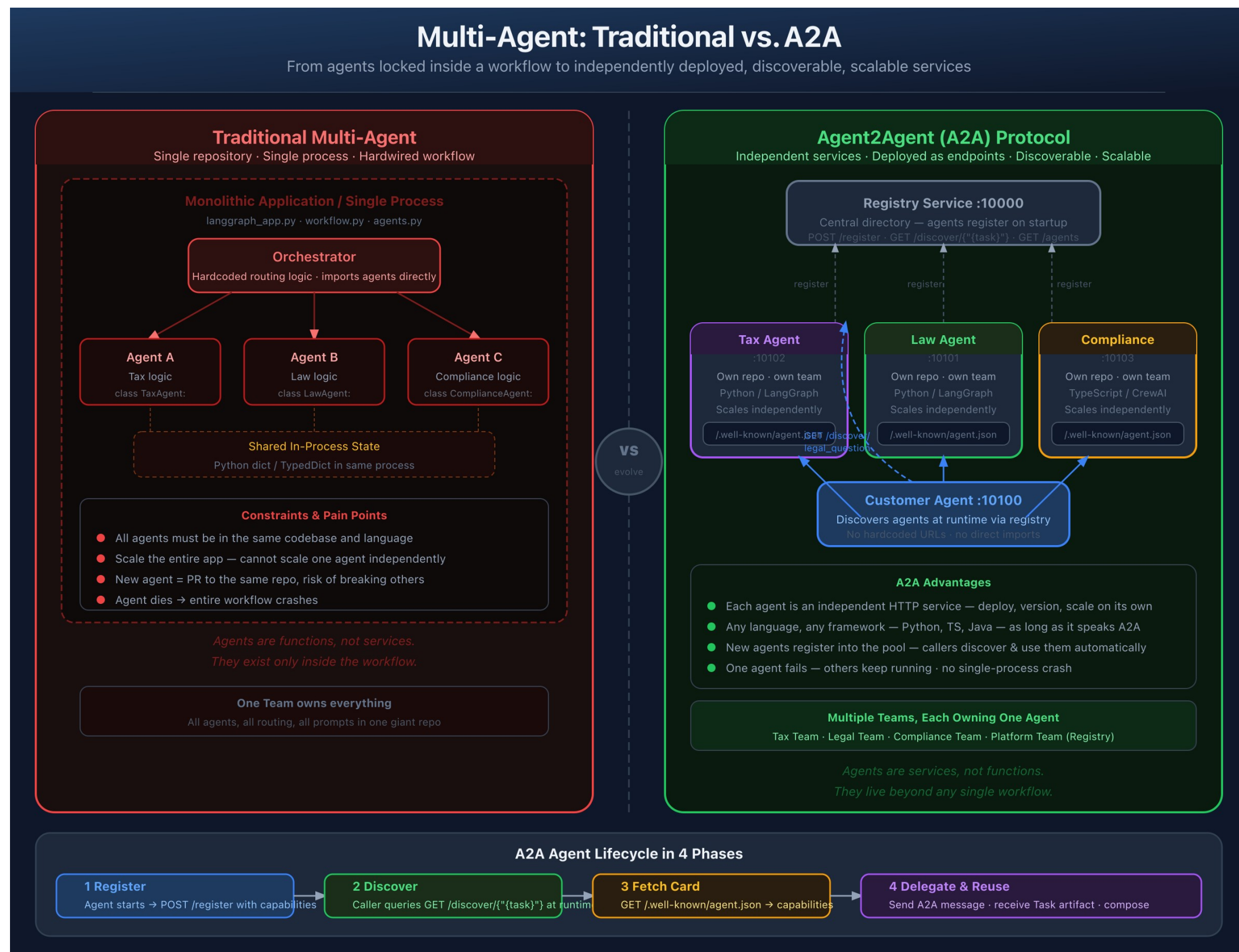


AgentCard • Task • Message • Part • Artifact • Context



# Multi-Agent: Traditional vs A2A

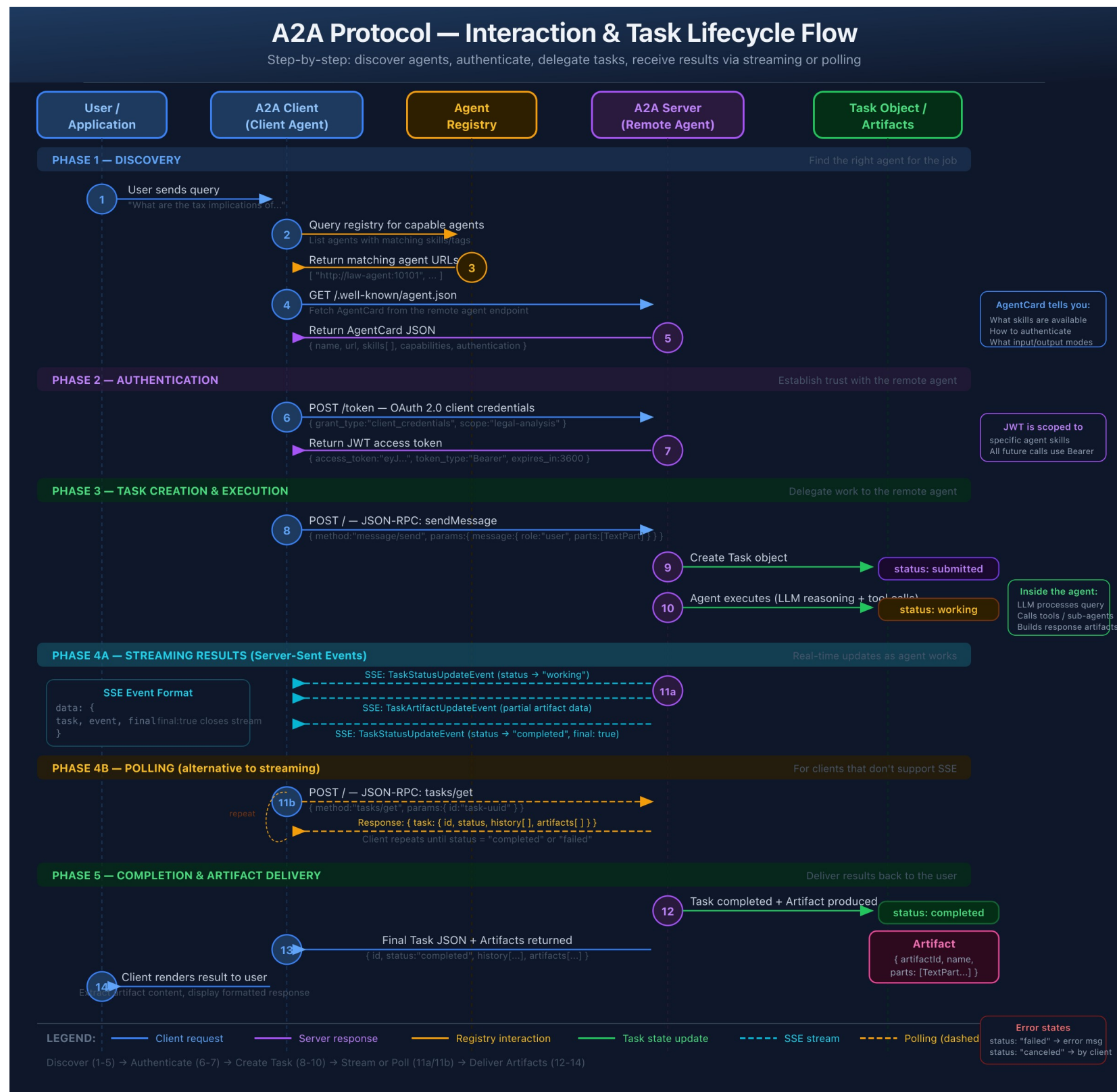
Từ in-process sang HTTP services độc lập



# A2A Interaction & Task Lifecycle

AI IN ACTION  
Đào tạo Nhân tài AI thực chiến

Discover → Authenticate → Delegate → Stream → Complete



# A2A vs MCP — So sánh có hệ thống

Hai chuẩn không cạnh tranh — chúng giải quyết hai bài toán khác nhau

|                   | MCP — Model Context Protocol              | A2A — Agent-to-Agent Protocol                    |
|-------------------|-------------------------------------------|--------------------------------------------------|
| Tổ chức công bố   | Anthropic (2024)                          | Google (2025)                                    |
| Mục đích          | Kết nối LLM với công cụ và dữ liệu        | Kết nối các agent với nhau                       |
| Quan hệ           | Client – Server (bất đối xứng)            | Peer – Peer (đối xứng)                           |
| Đơn vị giao tiếp  | Resource, Tool, Prompt                    | Task, Message, Artifact                          |
| Tự chủ (autonomy) | Tool thụ động — chỉ thực thi khi được gọi | Agent chủ động — tự lập kế hoạch, gọi agent khác |
| State / Lifecycle | Stateless — 1 lệnh = 1 response           | Stateful Task: submitted → working → completed   |
| Discovery         | Cấu hình tĩnh trong host (mcp.json)       | Động qua AgentCard ở /.well-known/agent.json     |
| Transport         | stdio · SSE · HTTP (JSON-RPC 2.0)         | HTTP + JSON-RPC + Server-Sent Events             |
| Streaming         | Có (qua SSE)                              | Có — bắt buộc trong specification                |

✓ **Quy tắc nhớ:** MCP cho mọi thứ *không phải agent* · A2A cho mọi thứ *là agent*.

**Kết hợp thực tế** → một Customer Agent (A2A Server) bên trong dùng nhiều MCP server (filesystem · postgres · github), đồng thời uỷ thác qua A2A đến Law / Tax / Compliance Agent.

## A2A Protocol — In This Project

How our Legal Multi-Agent system uses A2A — AgentCard, Task lifecycle, Message structure, and flow

### What is A2A?

A2A is an open protocol for AI agents to:

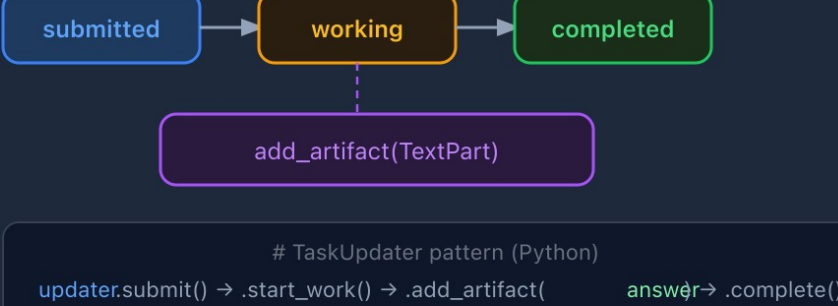
- Advertise their capabilities via an **AgentCard**
- Send and receive structured **Messages**
- Track long-running work as **Tasks**
- Stream partial results via **EventQueue**
- Remain model & framework neutral — any LLM, any stack

### AgentCard — The Agent's Identity

GET /.well-known/agent.json

```
{
  "name": "law-agent",
  "version": "1.0",
  "url": "http://localhost:10101",
  "skills": [ "legal_question" ],
  "capabilities": { "streaming": false }
}
```

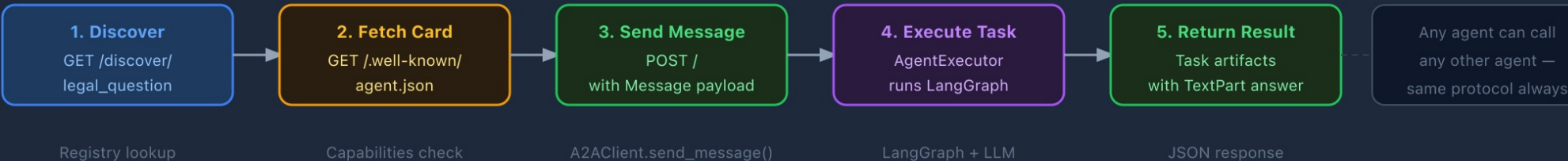
### Task Lifecycle



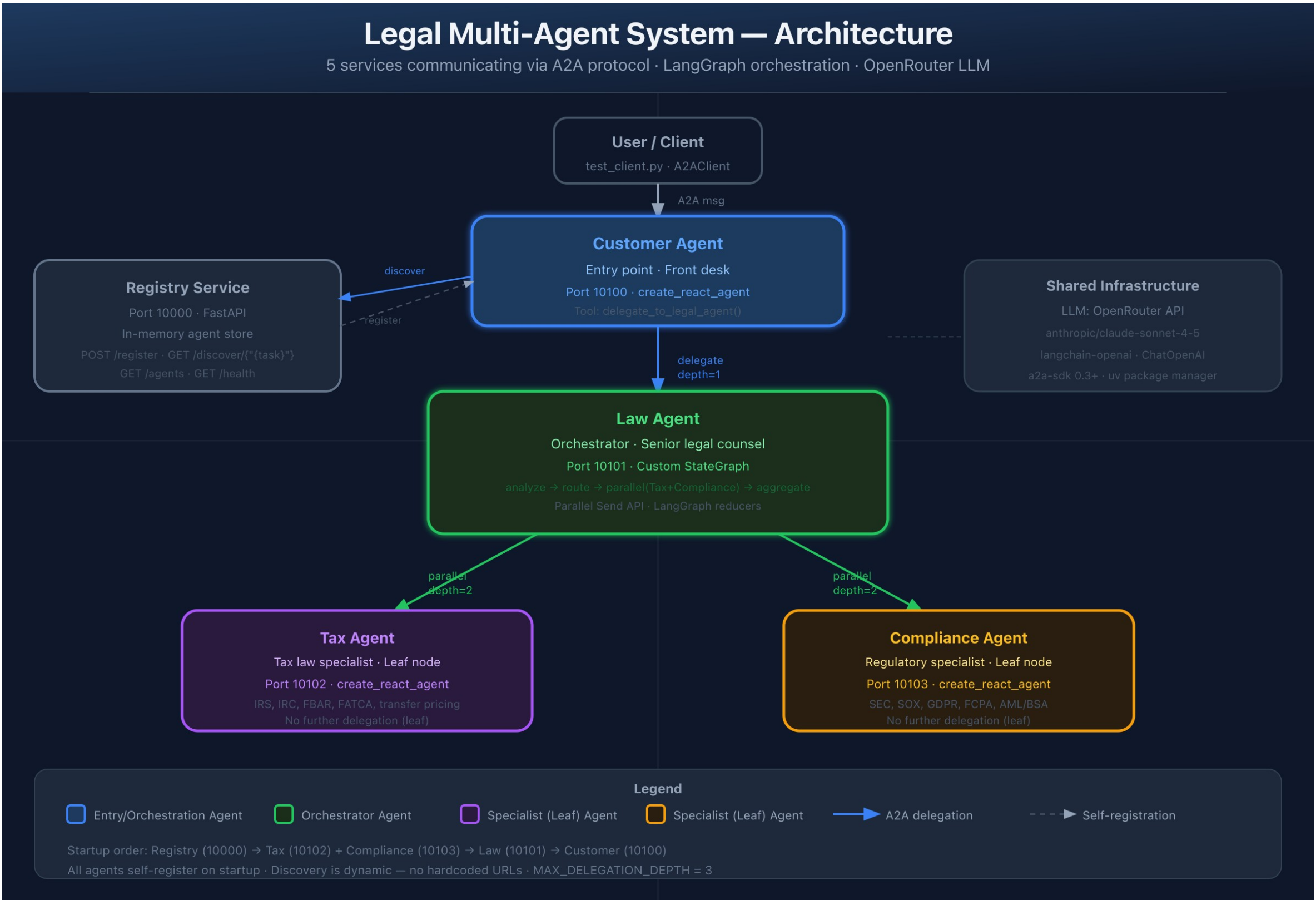
### Message Structure

```
{
  role: "user",
  message_id: "550e8400-e29b-41d4...",
  context_id: "ctx-abc123",
  parts: [ { TextPart: "Is tax evasion a crime?" } ],
  metadata: {
    trace_id: "trace-xyz", delegation_depth: 1
  }
}
```

### A2A Communication Flow in This Project



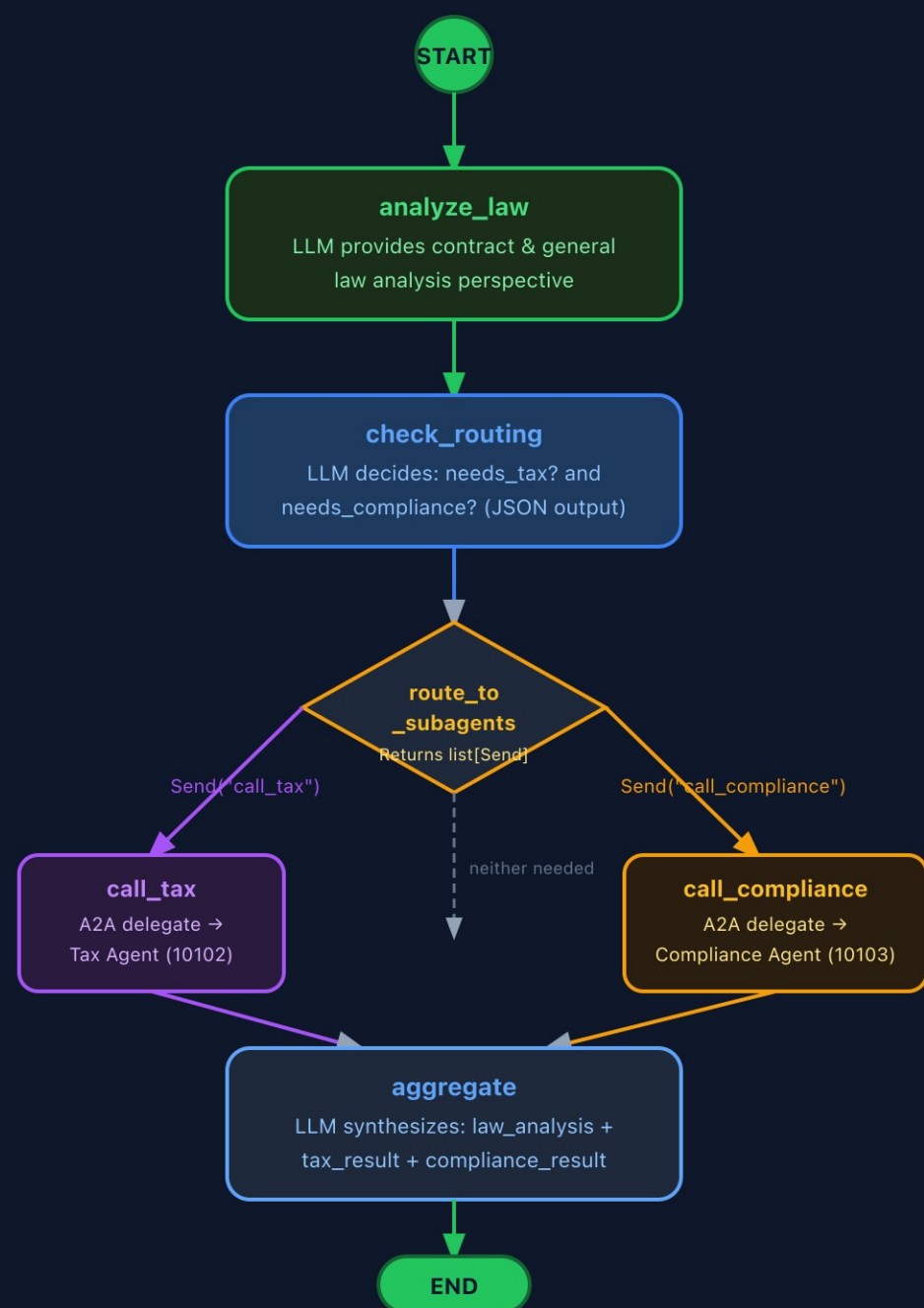
5 service • A2A + LangGraph • OpenRouter LLM



Parallel delegation • State merging • Depth guards

## Law Agent — LangGraph StateGraph Deep Dive

Parallel delegation via Send API • State merging with reducers • Depth-guarded recursion



### LawState — TypedDict with Reducers

```
class LawState (TypedDict):
    question: str
    law_analysis: str
    needs_tax: bool
    needs_compliance: bool
    tax_result: Annotated[str, _last_wins]
    compliance_result: Annotated[str, _last_wins]
    # reducer: parallel branches write safely
```

### LangGraph Send API — Parallel Branches

```
def route_to_subagents (state: LawState) -> list :
    targets = []
    if state["needs_tax"]: targets.append( Send("call_tax", state))
    if state["needs_compliance"]: targets.append( Send("call_compliance", state))
    return targets if targets else [Send("aggregate", state)]
# Both branches run concurrently — true parallelism inside LangGraph
```

### Delegation Depth Guard

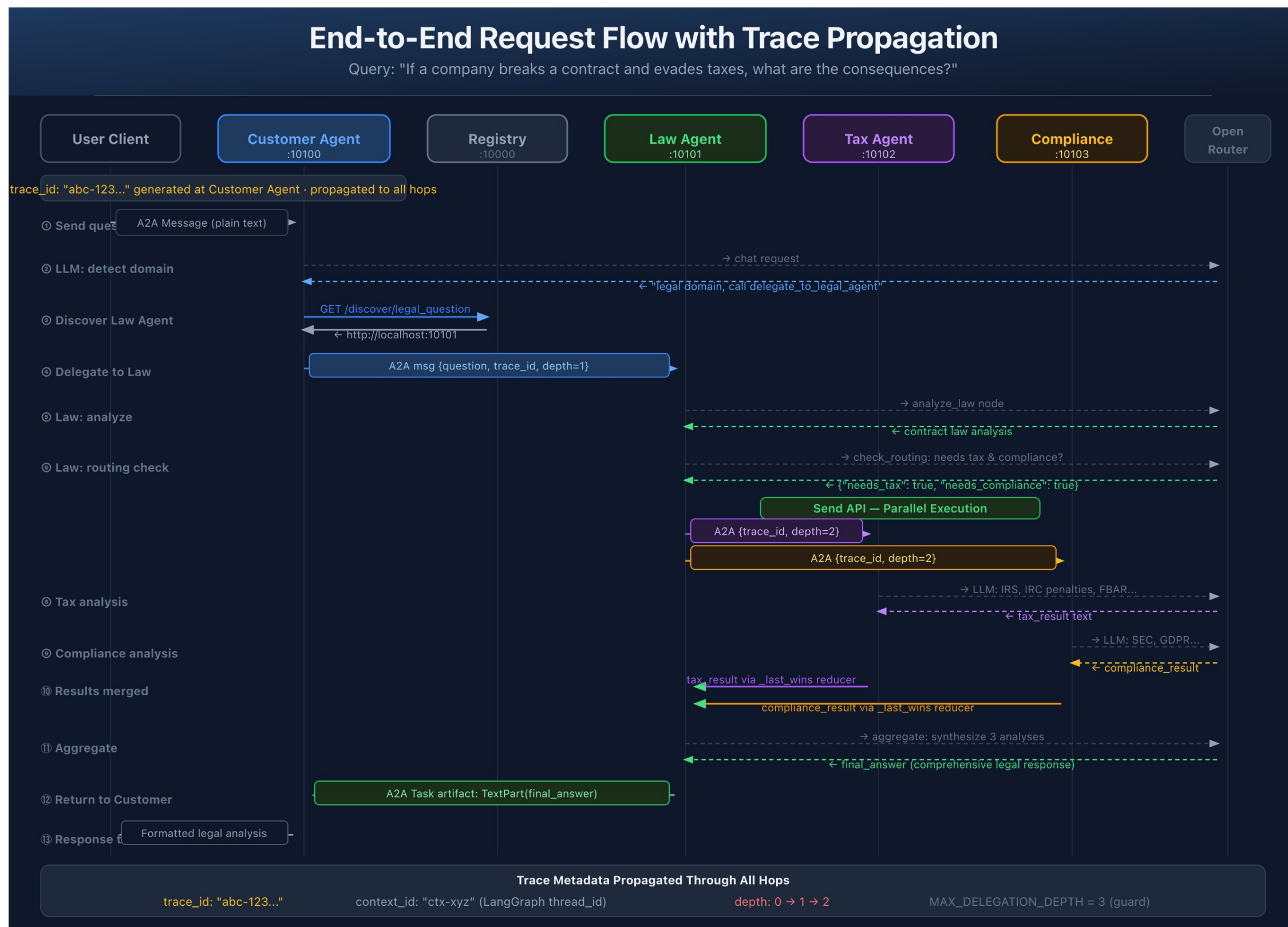
```
# Extracted from A2A message metadata at executor entry
depth = context.metadata.get( "delegation_depth" , 0)
if depth > MAX_DELEGATION_DEPTH # = 3
    state["needs_tax"] = False; state["needs_compliance"] = False
# Forces local-only answer — prevents infinite delegation loops
```

### Why LangGraph Send API over `asyncio.gather`?

- Proper LangGraph state management — reducers handle concurrent writes automatically
- Nodes are first-class graph nodes — visible in traces, debuggable in LangSmith
- Framework handles scheduling — cleaner than manual async task management

# End-to-End Request Flow

Theo dấu một câu hỏi xuyên qua 5 service



Những điểm cốt lõi cần ghi nhớ và bài tập về nhà

## 5 điểm cốt lõi (Take-aways)

1. LLM đơn lẻ → multi-agent là bước tiến tự nhiên khi domain phức tạp và cần song song hoá.
2. MCP chuẩn hoá cách LLM tiêu thụ công cụ và dữ liệu.
3. A2A chuẩn hoá cách các agent cộng tác như những peer độc lập.
4. Cả hai cùng tồn tại: A2A bên ngoài, MCP bên trong.
5. Observability bằng trace\_id là bắt buộc với hệ phân tán.

### Bài tập cá nhân (về nhà)

- Đọc spec A2A tại [github.com/google/A2A](https://github.com/google/A2A)
- Đọc spec MCP tại [modelcontextprotocol.io](https://modelcontextprotocol.io)
- Vẽ lại sơ đồ AgentCard của Law Agent từ `__main__.py`
- Đối chiếu StateGraph trong `law_agent/graph.py` với SVG 05

### Bài tập nhóm (project)

- Thêm một agent mới (vd. Finance Agent) — đăng ký với Registry
- Sửa Law Agent để uỷ thác sang Finance Agent khi câu hỏi về tài chính
- Bổ sung 1 MCP server (vd. filesystem) cho Tax Agent
- Báo cáo: vẽ sơ đồ + bảng so sánh hiệu năng có/không có parallel

## Tài liệu tham khảo

- A2A spec: [github.com/google/A2A](https://github.com/google/A2A)
- MCP spec: [modelcontextprotocol.io](https://modelcontextprotocol.io)
- LangGraph: [langchain-ai.github.io/langgraph](https://langchain-ai.github.io/langgraph)

? Hỏi - Đáp ?



# THANK YOU